CMPT 276 - Phase 2 Report (Milestone 1)

Project Group 14

Joshua Kim, Tim Yeh, Rodrigo Anasco, Sarah Herb

Our project is a 2D arcade-style game in which the player controls Doug, a hungry dog navigating a maze to collect food while avoiding enemies and punishments. The game consists of various maps, each with distinct rewards and punishments, depending on the map's theme (e.g., city, grassland). The objective is to collect all treats and reach the exit without losing to enemies or consuming punishments.

For this part of the project, we understood that our task was to bring our game design to life by implementing the classes outlined in our UML diagrams and updating the functionalities detailed in our use cases.

Approach to implement the game:

In approaching the implementation phase of our game, we began by covering as a team to discuss the specific tasks and goals set for this project stage. In this meeting we discussed what should be our final product for each milestone. Taking inspiration from Assignment 2, we decided to use the Factory Method design pattern, since it simplifies the creation of game objects like enemies, rewards, and mazes, allowing us to easily manage and expand upon these elements as our game develops.

Our class design builds directly from our initial UML diagrams. We established the following core classes: Doug, Enemy, Reward and Game Mechanics. Each class was divided into smaller tasks, for example, implementing Doug's movement and its interaction with objects, with enemies, rewards, the map, etc. This was a solid starting point, but we also discussed about the need of implementing new functions that were not in the UML diagram as the game required.

We divided the roles according to classes as below:

1. Doug: playable character,reward counter - Josh
2. Game mechanics, some UI - Sarah
3. Enemies: different types of enemies, obstacles - Tim
4. System: Obstacles and punishments - Rodrigo

For Milestone 1, our goal was to accomplish several foundational tasks in preparation for the next phase. First, we aim to complete portions of this report, focusing especially on content relevant to Milestone 1. This will include documenting our initial progress, setting the groundwork for Phase 2, and leaving room for further updates on changes, modifications, and challenges encountered along the way.

We also plan to implement the primary classes outlined in our UML diagrams. Our objective is to complete as much of the core implementation as possible within Milestone 1. This early groundwork will give us the flexibility to focus on refining and expanding key aspects in Milestone 2, such as the UI, map elements, exit points, game mechanics, and the main game loop. Recognizing that we may encounter scheduling constraints, we have

allowed ourselves some leeway for Milestone 2 if certain components cannot be fully completed by the midway point, though we expect this to be minimal.

For Milestone 2, our goal is to complete any additional functions necessary to fully satisfy the requirements outlined in our main UML diagram. With much of the core structure already implemented in Milestone 1, this phase will allow us to refine and finalize any remaining functionalities. Additionally, we plan to complete the report, building on the progress made in Milestone 1, and provide a comprehensive overview of our approach, modifications, challenges, and overall project development. By this stage, our work on both the code and report should be well-advanced, allowing us to focus on delivering a polished final product.

Adjustments and modifications from the initial design:

In our initial design, we envisioned specific classes and methods to manage game mechanics, character interactions, and map functionalities. However, during implementation, we made several adjustments to better align with practical needs and enhance gameplay flow.

1. Handler Class: We introduced the Handler class to streamline the management of all game objects, such as Doug, enemies, and items. The Handler class simplifies updating and rendering objects within the main game loop by centralizing these tasks. This design modification allowed us to handle object creation and deletion more flexibly.

2. Game Class: Our Game class serves as the primary controller, managing the main game loop, initializing game components, and controlling state changes like winning or losing. This class also incorporates random object generation (e.g., random rat or food placements), enabling us to create varied gameplay scenarios. Additionally, the game's background and environmental elements like bushes are rendered here, helping build a visually cohesive game environment.

3. KeyInput Class: This class manages keyboard inputs, allowing Doug's movement to be controlled with both the WASD keys. It addresses constraints like preventing diagonal movement, adding a smooth gameplay experience. The KeyInput class ensures player control is intuitive and prevents unintended directional conflicts.

4. MovingEnemy Class: Originally, enemies were conceptualized as simple objects; however, we introduced an abstract MovingEnemy class to allow for varied enemy behaviors (e.g., different chase strategies or penalties). The MovingEnemy class includes methods for interacting with Doug, applying penalties, and defining movement, laying a flexible foundation for creating diverse enemy types.

5. Enums: We implemented enums for ObstacleType and PunishmentType, which allowed us to categorize obstacles and punishments more effectively. This change made our code more modular and readable, as specific types like WALL, BUSH, ROTTEN_FOOD, ONION, and CHOCOLATE can now be referenced directly, streamlining game object creation and type checking.

In addition, while we initially intended to have a map class with cells on it, we actually managed spatial configurations and obstacles directly within Game and Handler. Environmental features like walls and bushes were placed in the render() method of Game.

External Libraries:

These libraries were chosen for their simplicity of usage and because it provides essential functionality for 2D game development

java.awt.Canvas: This class provides a drawing surface for rendering graphics in the game. By using Canvas, we have a flexible area where we can draw our game components and control how they appear on the screen, making it ideal for creating custom graphical interfaces like the game background and objects.

java.awt.Color: This library allows us to define colors for various game elements. With Color, we can set custom colors for Doug, enemies, and other objects, helping to visually differentiate them and enhance the overall aesthetic of the game.

java.awt.Graphics and java.awt.Graphics2D: These classes provide a graphics context for drawing shapes, images, and text on the Canvas. Graphics2D offers advanced drawing capabilities, like transformations and anti-aliasing, allowing us to render smoother and more visually appealing elements in the game

java.awt.image.BufferStrategy: This library enables double-buffering, which helps reduce flickering and improves rendering performance by using off-screen drawing buffers. Buffer strategies are essential in games to ensure smooth frame transitions and reduce screen tearing during gameplay.

java.awt.image.BufferedImage: BufferedImage is used for handling image data that can be rendered on the screen. This class allows us to load and display images like the game's background and character sprites, giving our game a more polished and visually engaging appearance.

javax.imageio.ImageIO: This class is used for reading and writing images. With ImageIO, we can load external image files (e.g., Doug, enemies, obstacles) and display them in the game. Using ImageIO enables flexibility in adding visual assets to the game, making it easier to include custom artwork and enhance the graphical quality.

java.util.Random: This class allows us to generate random numbers, which we use to randomise object placement and introduce variability in the game (e.g., positioning enemies or rewards).

Methods for ensuring code quality:

To ensure high-quality code, we employed multiple key principles, for example the usage of Enums, Encapsulation, JavaDocs, and Modularity, which collectively enhanced the readability and robustness of our project.

We used enums to improve the clarity of our code, this allowed us to categorise and manage game components more effectively. Enums helped us enforce consistency across the game and made it easy to add or adjust types without impacting other parts of the code.

We carefully encapsulated class data by setting private access to attributes that should not be exposed directly, such as Doug's position and health, or the specific properties of rewards and punishments.

To make our code more understandable and maintainable, we documented each class and method using JavaDocs. These comments provided descriptions, parameter explanations, and return values for each public method, making the purpose and usage of every component clear.

We structured our project in a modular way, breaking down functionality into specialized classes such as Doug, Handler, Game, and MovingEnemy. By doing so, we improved code reusability and minimized dependencies between components. For instance, the Handler class is responsible for updating and rendering game objects, while Doug handles player-specific behavior.

These principles allowed us to build a clean and scalable codebase, improving our development workflow and ensuring that our code is accessible, adaptable, and aligned with best practices in software engineering.

Challenges:

One of the biggest challenges we faced during this phase was managing version control and file merging on GitHub. With multiple team members working on different components of the project simultaneously, we encountered conflicts when merging changes, especially with core files that several team members needed to modify. Resolving these conflicts required careful attention to avoid overwriting each other's work, and at times, we had to rebase and review the changes line by line to ensure consistency. This process was time-consuming and occasionally disrupted our workflow, but it ultimately strengthened our collaborative practices and taught us valuable lessons in effective version control management.

Another significant challenge was time management, particularly due to the overlap with the midterm period for some of our team members. Balancing project development with studying for exams created scheduling conflicts and slowed our progress at times. We had to coordinate around each member's availability and adjust our timeline accordingly, occasionally leading to delays in planned milestones.

References Used

Dog and Rat Sprites:
https://free-game-assets.itch.io/free-street-animal-pixel-art-asset-pack?download
– Used for the main character (Doug) and enemy (rat) sprites.

Food Sprites: https://henrysoftware.itch.io/pixel-food – Used for various collectible food items as rewards in the game.

Game Mechanics and Backend Setup: https://www.youtube.com/watch?v=1gir2R7G9ws – Provided foundational guidance for the game loop and object management structure.